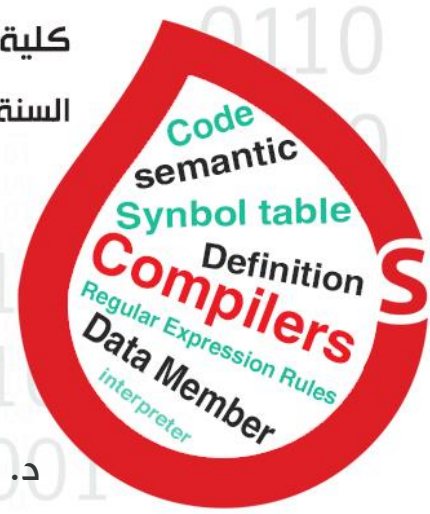




Top Down Parsing



05/11/2023

د. باسم قصيبة

محتوى مجاني غير مخصص للبيع التجاري

RB Informatics;

المترجمات

قبل البدء بالمحاضرة سنقوم بتذكّر بعض المعلومات والملاحظات ممّا سبق , لنبدأ على بركة الله ♥

1. شجرة الـ AST هي بنية معطيات خارجية يتم توليدها من الـ Parser
2. يوجد طريقتين لعمل compiler و interpreter :
 - **Listener**: تعتبر هذه الطريقة صعبة بسبب صعوبة توليد شجرة الـ AST وذلك لأنها تعتمد على العودية في توليد العقد لذلك نلجأ إلى الطريقة الثانية.
 - **Visitor**: تعتبر الطريقة الأسهل
3. ليس ضروري أن تكون شجرة الـ AST مطابقة تماماً للكود , فمثلاً من الممكن الاستغناء عن الأقواس ضمن شجرة الـ AST
4. الـ if, while, do while تسمى Block حيث يتكوّن الـ Block من 2 Data number (2 nodes):
 - **العقدة الأولى** عبارة عن Expression (أي شرط يعبر عن علاقة بين عقدتين)
 - **العقدة الثانية** عبارة عن statement حيث:

$statement \rightarrow statement \mid \epsilon \rightarrow \text{Null}$
 لا شيء
 Statement جديدة يتم تنفيذها

5. مرحلة الـ semantic وتوليد الكود غير مطلوبة في مشروع الفصل الأول وكذلك الـ error handling
6. بعد توليد شجرة الـ AST يتم توليد الكود وكذلك check Type وهذا ما يدعى الـ translation حيث عملية توليد الكود تتم على كل عقدة على حدى وكل عقدة تولد جزء من الكود وبعدها يتم استدعاء ابنائها أمّا الـ check type فتقوم بالتحقق من نوع العقدة قبل إضافتها إلى symbol table حيث إذا كان هناك عملية تؤثر على الـ type يتم إجرائها وكذلك عملية التحقق من العقد المترابطة الذي يجب أن تكون من نفس النوع وإلا يحدث error.
7. أنواع القواعد في الـ compiler:

- **قواعد بسيطة** : بدائلها عبارة عن terminal مثل: $Inst \rightarrow a \mid b$
- **قواعد مركبة**: بدائلها عبارة عن non terminal مثل :

$seq \rightarrow seq Inst$
 $Inst \rightarrow a \mid b$

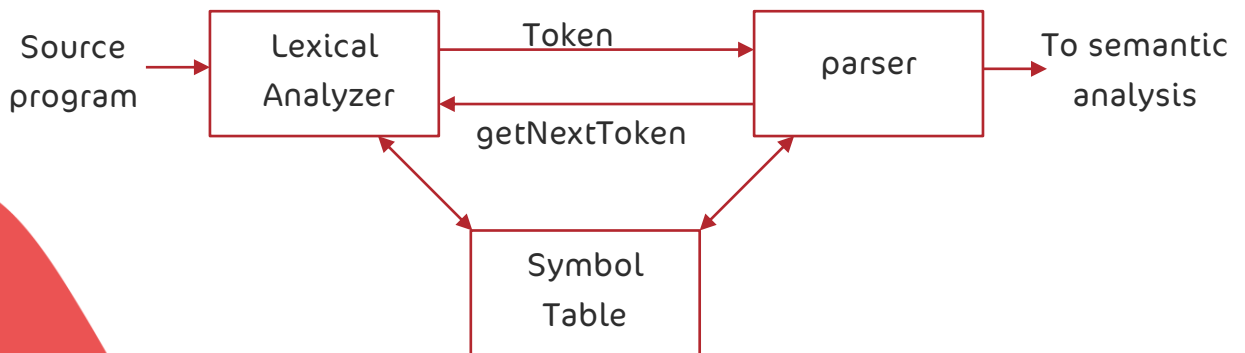


المحلل اللفظي Lexical Analyzer

هي أولى مراحل تصميم الـ Compiler يقوم بتجزئة النص البرمجي إلى سلسلة محارف عبارة عن tokens يتم توليده بشكل يدوي ، أي يتم كتابة كود للتعرف على المفردات ضمن لغة ما وإعادة معلومات عنها ويقوم الـ antlr بهذه العملية حيث تعتبر هذه الطريقة الأسرع لأن توصيف مفردات اللغة تكون على مستوى عالي من التجريد وكذلك تعتبر أسهل للتعديل..

1. العلاقة بين المحلل اللفظي والـ Parser :

يقوم المحلل اللفظي بتزويد الـ parser بالـ tokens وذلك لتوليد شجرة الـ AST ثم يطلب الـ parser من المحلل اللفظي token جديد.



2. وظائف المحلل اللفظي

1. حذف الفراغات والتعليقات (scanning)

2. إنتاج سلسلة الـ tokens إلى الـ parser

3. المفردات التي يتعرف عليها المحلل اللفظي

1. الـ keywords (الكلمات المحجوزة في اللغة) مثل if , for , while , int...

2. الـ operator: مثل = , > , < , ≤

3. الـ Identifiers (أسماء المتحولات والكلاسات و الباكجات) ويتم التعرف عليها من خلال الـ regular expression

4. الثوابت: مثل الأرقام والـ string وكذلك يتم التعرف عليها من خلال الـ Regular expression

5. Punctuation symbol مثل الأقواس والفاصلة المنقوطة والفواصل

الفرق بين الـ tokens والـ pattern:

الـ token هو عبارة عن زوج من نوع type وقيمة value وسلسلة محارف تدعى lexeme

الـ pattern عبارة عن الكلمات المحجوزة في اللغة مثل if , for , class...



ملاحظات:

- الـ Regular Expression على الرغم من بساطتها إلا أنها مهمة جدا في تصميم الـ compiler.
- ليست كل سلسلة من الـ tokens تمثل برنامج يجب أن يميز الـ parser بين سلسلة الـ tokens الصالحة وغير الصالحة لذلك فإننا نحتاج للغة تصف السلاسل الصالحة أي تحتاج لطريقة للتمييز بين سلاسل الـ tokens الصالحة وغير الصالحة.
- نقول عن سلسلة من الـ tokens أنها صالحة عندما تكون الـ parser Tree الموافقة لها صحيحة وتكون الـ parse Tree صحيحة عندما يكون شكل الشجرة مطابق لـ syntax اللغة التي تعمل عليها أي مطابق لقواعد التعرف على هذه اللغة CFG لذلك فالهدف من الـ Parse Tree هي معرفة إن كانت سلسلة الـ tokens المدخلة تشكل قاعدة صحيحة .

تمثيل الـ Regular Expression

- ❖ محرف واحد $C' = \{ "C" \}'$
- ❖ إيبسلون $\{ "" \} = \epsilon$
- ❖ الاجتماع $A + B = \{ S \mid S \in A \text{ or } S \in B \}$
- ❖ الوصل $AB = \{ ab \mid a \in A \text{ and } b \in B \}$
- ❖ التكرار $A^* = \cup_{i \geq 0} A^i$ where $A^i = A \dots i \text{ times } \dots A$

Syntax & Semantics in RE

- ❖ $L(\epsilon) = \{ "" \}$
- ❖ $L(C') = \{ "C" \}$
- ❖ $L(A + B) = L(A) \cup L(B)$
- ❖ $L(AB) = \{ ab \mid a \in L(A) \text{ and } b \in L(B) \}$
- ❖ $L(A^*) = \cup_{i \geq 0} L(A^i)$

أمثلة على استخدام الـ RE في الـ Compiler

1. Identifier في لغة برمجة: نحن نعرف أنه في جميع لغات البرمجة لا يمكن البدء برقم أو رمز أي يجب أن يبدأ الـ Identifier بحرف أو underscore عند تعريف Identifier فتكون القواعد:

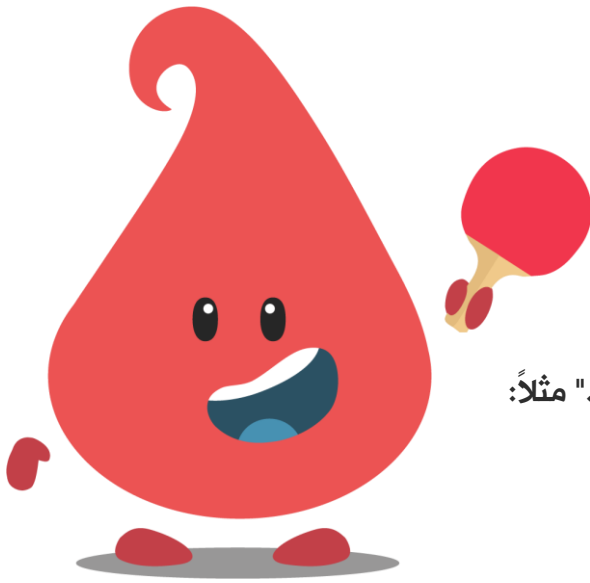
$$\begin{aligned} \text{letter_} &\rightarrow A \mid B \mid \dots \mid Z \mid a \mid \dots \mid z \mid _ \\ \text{digit} &\rightarrow 0 \mid 1 \mid 2 \mid \dots \mid 9 \\ \text{Id} &\rightarrow \text{letter_}(\text{letter_}|\text{digit})^* \end{aligned}$$

2. لتعريف عدد integer أو float أو رقم باسكال أي أحد الأشكال (52 / 0.12 / 6.3E4 / 6.8E-4) :

$$\begin{aligned} \text{digit} &\rightarrow 0 \mid 1 \mid 2 \mid \dots \mid 9 \\ \text{digits} &\rightarrow \text{digit}^* \\ \text{optinal Fraction} &\rightarrow \text{digits}|\epsilon \\ \text{optional Exponent} &\rightarrow (E(+|-|\epsilon)\text{digits})|\epsilon \\ \text{number} &\rightarrow \text{digits optionalFraction optionalExponent} \end{aligned}$$


بعض الرموز المستخدمة في antlr ومعناها في Regular Expression:

- $operator^+$ نسخة واحد أو أكثر
- $operator^*$ صفر نسخة أو أكثر
- $operator^?$ صفر أو نسخة واحدة
- $[abc] = a|b|c$
- $[a - z] = a|b|c| \dots |z$
- $r\{m, n\}$ تكرار r على الأقل m مرة وعلى الأكثر n مرة
- $[^S]$ أي حرف عدا S
- $'.'$ تشير إلى محرف واحد (أي محرف ممكن أن يعوض بدلا من "." مثلاً: $c.r$)
- ممكن بدلا من $a || b || c || \dots$ أن يكون $a || b || c || \dots$
- $^$ تشير إلى بداية السطر
- $\$$ تشير إلى نهاية السطر
- $\backslash c$ تشير إلى أن C محرف خاص , مثل $\backslash n$ في $c++$



أمثلة على استخدام القواعد السابقة

1. للتعبير عن Identifier باستخدام القواعد

$$Id \rightarrow [A - Z a - z _][A - Z a - z _ 0 - 9]^*$$

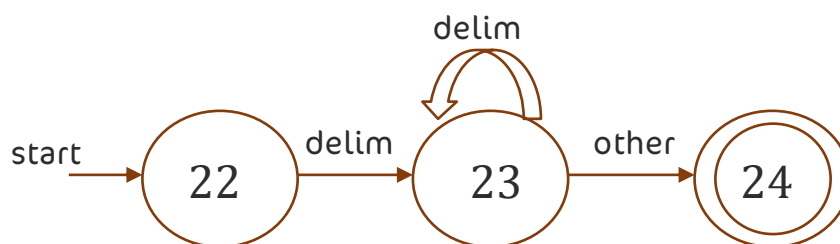
2. للتعبير عن أعداد باسكال

$$digit \rightarrow [0 - 9]$$

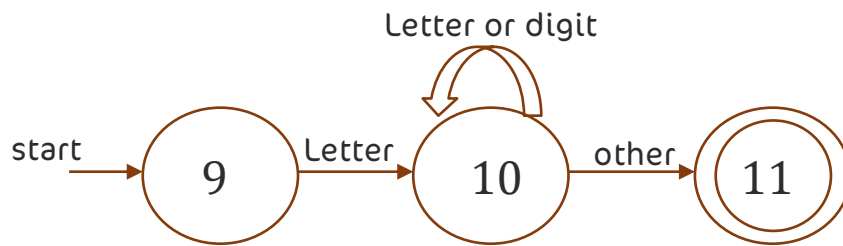
$$digits \rightarrow digit^+$$

$$number \rightarrow digits(. digits)? (E[+ -]? digits)?$$

بعض الأوتومات المصممة للتعرف على token



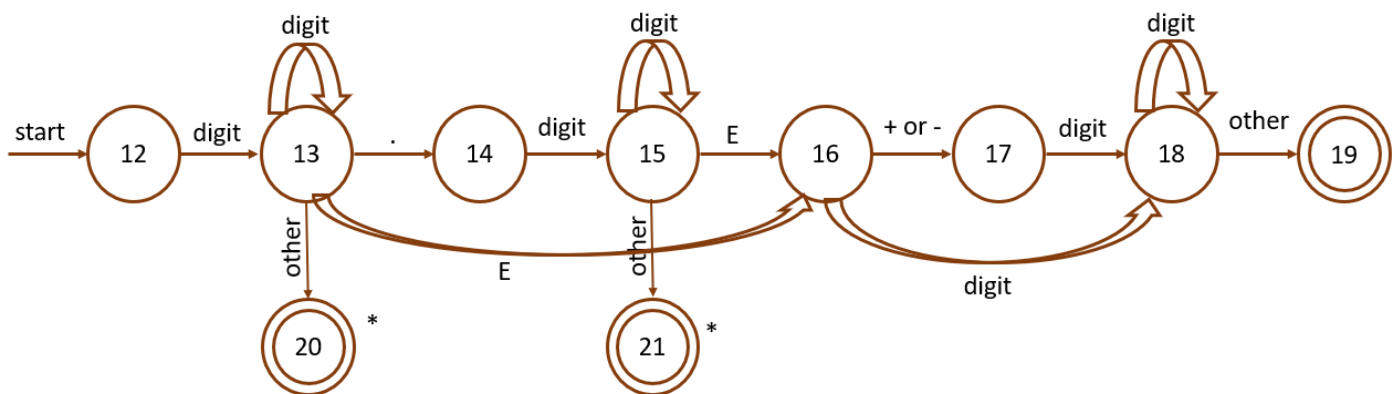
Recognition of white spaces



Recognition of identifiers



Recognition of then



Recognition of number

Automate1

```

TOKEN getRelop(){
    TOKEN retToken = new(RELOP);
    While(1){
        Swich(state) {
            case0: c = nextchar();
            if(c = '<') state = 1;
            else if(c = '=') state = 5;
            else if (c = '>') state = 6;
            else fail();
            break;
            case1: ...
            case8: retract();
                retToken.attribute = GT;
                return(retToken);
            break;
        }
    }
}
    
```

Automate2

```

state ← 0; start ← 0;
lexecal value;
int fail(){
    forward = token_beginning;
    switch(start){
        case0: state ← 9; break;
        case9: start ← 12; break;
        case12: start ← 20; break;
        case20: start ← 25; break;
        case25: recover;
    }
}
    
```



توضيح الكود السابق:

بداية تكون الـ $state=0$ فيدخل إلى الـ $case0$ فيطلب قراءة محرف

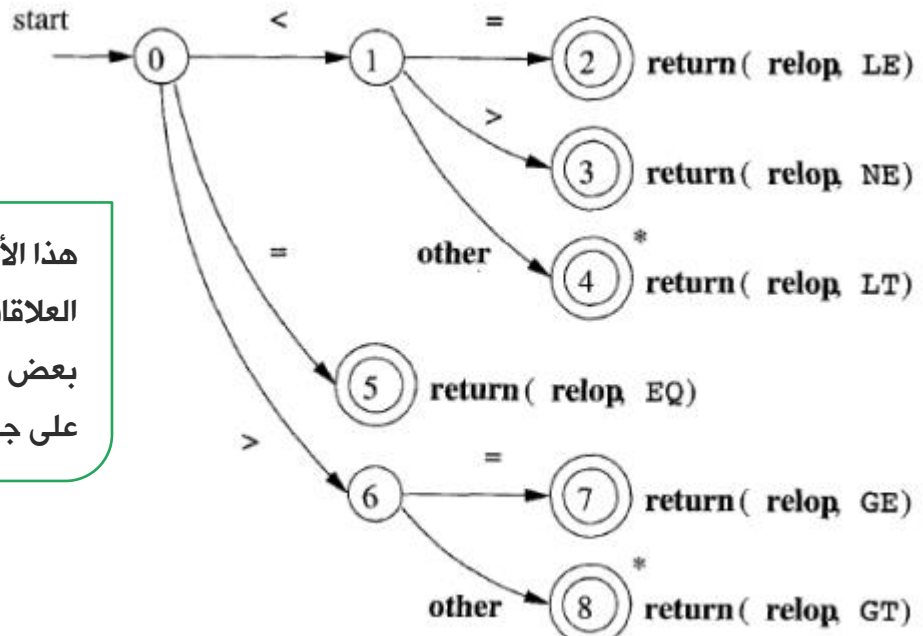
إذا كان المحرف ' $<$ ' ينتقل إلى الـ $state=1$

وإذا كان '=' ينتقل إلى الـ $state=5$

وإذا كان ' $>$ ' ينتقل إلى الـ $state=6$ وإلا يقوم باستدعاء أوتومات آخر يدعى $fail()$ مهمة هذا الأوتومات التعرف على أنواع أخرى غير الـ $relation$ وذلك عندما يفشل الـ $lexcer$ في التعرف على الـ $tokens$ ويستمر بالمحاولة بالتعرف على نوع آخر وفي حال قام بتجريب جميع الحالات و فشل يقوم بعمل $recover$ (ستشرح في المحاضرات القادمة)

ولكن باختصار مهمتها تجاوز الأخطاء التي حدثت بسبب عدم التعرف على أي نوع

نلاحظ في الكود الأول في الـ $case8$ يتم استدعاء تابع $retract$ وظيفته باختصار إرجاع آخر رمز ومعالجته (أي نقله من الـ $parser$ إلى الـ $lexer$).



هذا الأوتومات مصمم للتعرف على العلاقات ويمكن أن يضاف إليه بعض الأوتومات الأخرى ليتعرف على جميع الرموز



Recognition of relation

4. من وظائف محلل القواعد Syntax analyzer:

1. استقبال الـ $tokens$ من محلل المفردات من قِبَل الـ $Parser$.
2. التحقق فيما إذا كان من الممكن اشتقاق هذه السلسلة بدءاً من الـ $symbol table$ وذلك بواسطة الـ $parser$.
3. الـ $recovering$ أي متابعة التنفيذ على الرغم من وجود أخطاء , مثلاً: ظهر خطأ في السطر رقم 15 عندئذٍ الـ $compiler$ لا يتوقف و إنما يحاول إظهار بقية الأخطاء.

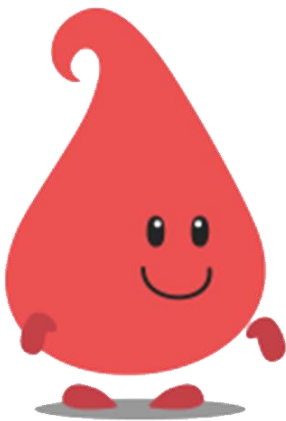
ملاحظة: خوارزمية CYK تقوم بتجميع العقد من العقد الأوراق وصولاً إلى الجذر بتعقيد أُسّي لذلك لا يمكن الاعتماد عليها للإنتاج محلل قواعدي وتعتبر هذه الخوارزمية من نوع Bottom Up سنتعرف عليه لاحقاً.

الفرق بين الـ top down والـ Bottom up :

الـ top down: تسمح سلسلة الرموز من اليسار إلى اليمين وتقوم ببناء شجرة الإعراب من الجذر إلى الأوراق وهذه الأوراق يجب أن تكون مكافئة للمفردات التي لدي.

الـ Bottom up: تسمح سلسلة الرموز من اليسار إلى اليمين وتبدأ من الأوراق وعند الوصول إلى رمز البداية يجب أن تكون قد مسحت سلسلة المفردات وإلا هناك خطأ.

اشتقاق اللغات خارج السياق والأوتومات بمكدس



■ نقول أن عملية الاشتقاق قد انتهت عندما ينتهي الدخل ويكون المكدس فارغ

■ نرمز لعملية الاشتقاق بالرمز δ ويكون لتابع الانتقال الشكل التالي:

$$\delta(s_0, a, b) \rightarrow (f, s)$$

حيث: s_0 رمز الحالة الحالية و s ما سيتم إدخاله بالمكدس

f رمز الحالة التالية

a هو الدخل و b هو top of stack

■ في البداية يكون المكدس فارغ لذلك ندخل إليه start symbol

مثال على الاشتقاق:

لدينا القواعد خارج السياق التالية $\{S \rightarrow aSa | bSb | \varepsilon\}$ ونريد إعراب الجملة $abba$:
قواعد الانتقال للأوتومات بمكدس المكافئ لها كالآتي:

- $\delta(s_0, \varepsilon, \varepsilon) \rightarrow (f, S)$
- $\delta(f, \varepsilon, S) \rightarrow (f, aSa)$
- $\delta(f, \varepsilon, S) \rightarrow (f, bSb)$
- $\delta(f, \varepsilon, S) \rightarrow (f, \varepsilon)$
- $\delta(f, a, a) \rightarrow (f, \varepsilon)$
- $\delta(f, b, b) \rightarrow (f, \varepsilon)$

الرمز \$ يشير إلى نهاية

الstring

State	input	stack	δ
s_0	\$abba	\$	1
f	\$abba	S\$	2
f	\$abba	aSa\$	5
f	\$abb	Sa\$	3
f	\$abb	bSba\$	6
f	\$ab	Sba\$	4
f	\$ab	ba\$	6
f	\$a	a\$	5
f	\$	\$	

شرح المثال السابق

■ بدايةً نقوم باستخراج توابع الانتقال عند القواعد:

$$\delta(S_0, \varepsilon, \varepsilon) \rightarrow (f, S)$$

❖ أي عندما يكون المكّس فارغ نضيف فيه الـ start symbol وننتقل إلى الحالة f :

$$\delta(f, \varepsilon, S) \rightarrow (f, aSa)$$

$$\delta(f, \varepsilon, S) \rightarrow (f, bSb)$$

❖ أي عندما يكون لدينا في قمة المكّس الرمز S نستبدله بـ aSa أو bSb وننتقل إلى الحالة f :

$$\delta(f, \varepsilon, S) \rightarrow (f, \varepsilon)$$

$$\delta(f, a, a) \rightarrow (f, \varepsilon)$$

$$\delta(f, b, b) \rightarrow (f, \varepsilon)$$

❖ أي عندما يكون في قمة المكّس S وكان الدخل قد انتهى نحذف S وعندما يكون في قمة المكّس a والدخل

الحالي a أي حذف تطابق نحذف قمة المكّس , وعندما يكون في قمة المكّس b والدخل الحالي b أي حدث

تطابق نحذف قمة المكّس

- $\delta(s_0, \varepsilon, \varepsilon) \rightarrow (f, S)$
- $\delta(f, \varepsilon, S) \rightarrow (f, cAd)$
- $\delta(f, \varepsilon, A) \rightarrow (f, ab)$
- $\delta(f, \varepsilon, A) \rightarrow (f, a)$
- $\delta(f, a, a) \rightarrow (f, \varepsilon)$
- $\delta(f, b, b) \rightarrow (f, \varepsilon)$
- $\delta(f, c, c) \rightarrow (f, \varepsilon)$
- $\delta(f, d, d) \rightarrow (f, \varepsilon)$

مثال آخر على الاشتقاق:

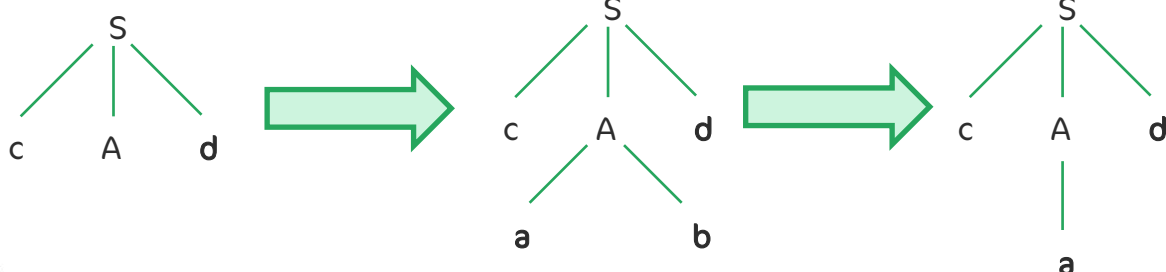
لدينا القواعد: $(S \rightarrow cAd)$ و $(A \rightarrow ab|a)$ ونريد إعراب الجملة cad :

انتقالات الأوتومات بمكّس كالتالي

State	input	stack	δ
S_0	$cad\$$	$\varepsilon\$$	1
f	$cad\$$	$S\$$	2
f	$cad\$$	$cAd\$$	7
f	$ad\$$	$Ad\$$	4
f	$ad\$$	$ad\$$	5
f	$d\$$	$d\$$	8
f	$\varepsilon\$$	$\varepsilon\$$	

هنا عندما أصبح محتوى المكّس Ad يمكن استبداله إما بـ ab أو بـ a وهذا ما يسمى بعدم القدرة على اتخاذ القرار وذلك لأن استخدام مفهوم $first$ وحده أصبح غير كافٍ لذلك ظهر مفهوم آخر هو $follow$ أي باختصار مهمته معرفة ما يتبع القاعدة التي سببت عدم القدرة على اتخاذ القرار لذلك نقوم بالنظر إلى ما يتبعها

في مثالنا كان الدخل ad ومحتوى المكّس Ad بالنظر إلى ما يتبع A نجد أنها d فنجد أن القيمة المناسبة لـ A هي a وليس ab :



الFirst: مجموعة الterminal الذي يبدأ بها القاعدة سواء بشكل مباشر أو غير مباشر
الFollow: مجموعة الterminal التي تتبع القاعدة مباشرةً.

Predictive Parsing

تتميز هذه القاعدة بأنها قادرة على التنبؤ بالقاعدة التي سيتم اختيارها لاستبدال سلسلة الدخل وعدم استخدام التراجعية، أي القدرة على توقع الطريق الذي يؤدي إلى سلسلة الدخل عن طريق معرفة الرموز التالية next tokens بواسطة الlook ahead حيث تستخدم مؤشر يُوْشر إلى الرمز التالي في سلسلة الدخل.
إنّ هذا النوع من الparsing لا يقبل إلا القواعد التي من النمط $ll(k)$ حيث:

- تشير الـ الأولى إلى طريقة المسح (القراءة) حيث تتم من اليسار إلى اليمين left-to-right .
 - تشير الـ الثانية إلى أن الاشتقاق يساري leftmost derivation كل شيء قبل النقطة الحالية هو terminals وكل شيء بعدها هو خليط من الterminal والnon-terminal .
 - تشير الـ k إلى عدد الـtokens التي سيتم أخذها بواسطة مؤشر الlook ahead .
- لكي يتم إجراء تحليل للقواعد بواسطة الpredictive Parsing يجب أن يكون عدد المحارف $k=1$.

1. قواعد إنتاج الFirst

- ❖ $FIRST(X)$
- ❖ If X is terminal $\rightarrow FIRST(X) = \{X\}$
- ❖ If $X \rightarrow^* \varepsilon \rightarrow FIRST(X) = FIRST(X) \cup \{\varepsilon\}$
- ❖ If $X \rightarrow \alpha_1 | \alpha_2 | \dots | \alpha_n \rightarrow FIRST(X) = FIRST(\alpha_1) \cup FIRST(\alpha_2) \cup \dots \cup FIRST(\alpha_n)$
- ❖ If $X \rightarrow ABC \rightarrow FIRST(X) = FIRST(ABC)$
 - If ε doesn't in $FIRST(A) \rightarrow FIRST(X) = FIRST(A)$
 - If ε in $FIRST(A) \rightarrow FIRST(X) = FIRST(A) - \{\varepsilon\} \cup FIRST(BC)$
and we compute $FIRST(BC)$ as above.

2. قواعد إنتاج الFollow

- ❖ $FOLLOW(X)$
- ❖ If X is start symbol $\rightarrow FOLLOW(X) = \{\$ \}$
- ❖ If $A \rightarrow \alpha X \beta \rightarrow FOLLOW(X) = FIRST(\beta) \text{ except } \{\varepsilon\}$
- ❖ If $A \rightarrow \alpha X \beta$ and $\beta \rightarrow \varepsilon \rightarrow$ add $FOLLOW(A)$ to $FOLLOW(X)$
- ❖ If $A \rightarrow \alpha X \rightarrow$ add $FOLLOW(A)$ to $FOLLOW(X)$

3. أمثلة على استخدام الFirst والFollow:

Productions....

- $E \rightarrow TE'$
- $E' \rightarrow +TE' | \varepsilon$
- $T \rightarrow FT'$
- $T' \rightarrow^* FT' | \varepsilon$
- $F \rightarrow (E) | id$

First....

- $First(E) = \{ (, id \}$
- $First(E') = \{ +, \varepsilon \}$
- $First(T) = \{ (, id \}$
- $First(T') = \{ *, \varepsilon \}$
- $First(F) = \{ (, id \}$

Follow....

- $Follow(E) = \{ \$,) \}$
- $Follow(E') = \{ \$,) \}$
- $Follow(T) = \{ \$,) , + \}$
- $Follow(T') = \{ \$,) , + \}$
- $Follow(F) = \{ *, \$,) , + \}$

شرح المثال السابق

- 1) $E \rightarrow TE'$
- 2) $E' \rightarrow +TE' | \varepsilon$
- 3) $T \rightarrow FT'$
- 4) $T' \rightarrow * FT' | \varepsilon$
- 5) $F \rightarrow (E) | id$



$$First(E) = \{ (, id \}$$

نلاحظ من القاعدة (1) أن $First(E)$ هو نفسه $First(T)$ و $First(T)$ هو نفسه $First(F)$

حيث: $First(F) = \{ (, id \}$

$$First(E') = \{ +, \varepsilon \}$$

من القاعدة (2) نلاحظ أن $+$ هي $first$ و أيضاً وجود ε بشكل مباشر أيضاً لذلك تطبق $\{ +, \varepsilon \}$ فنلاحظ أن $first$ يمكن أن تحوي على ε

$$First(T) = \{ (, id \}$$

نلاحظ من القاعدة (3) أن $First(T)$ هو نفسه $First(F)$

$$First(T') = \{ *, \varepsilon \}$$

من القاعدة (2) نلاحظ أن $*$ هي $first$ و أيضاً وجود ε بشكل مباشر أيضاً لذلك تطبق $\{ *, \varepsilon \}$

$$First(F) = \{ (, id \}$$

بشكل مباشر أن $First(F)$ هو $\{ (, id \}$



تنمة شرح التمرين السابق

$$Follow(E) = \{ \$,) \}$$

نلاحظ أن القاعدة E هي Start Symbol لذلك نضيف $\$$ وكذلك نلاحظ ظهور E في القاعدة (5) لذلك نضيف $)$.

$$Follow(E') = \{ \$,) \}$$

نلاحظ ظهور E' في القاعدة رقم (1) وحسب القاعدة $A \rightarrow aX$ (add Follow (A) to Follow (X))

$$Follow(T) = \{ \$,), + \}$$

من القاعدة رقم (1) نلاحظ أن E' يمكن أن تكون ε لذلك نضيف $Follow(E)$ إلى $Follow(T)$ ومن القاعدة رقم (2) ملاحظ أن $Follow(T)$ هو $Follow(T')$ لذلك نضيف $First(E')$ إلى $Follow(T)$

$$Follow(T') = \{ \$,), + \}$$

من القاعد (3) نلاحظ أم $Follow(T)$ نلاحظ أن $Follow(T')$ هو $Follow(T)$

$$Follow(F) = \{ \$,), +, * \}$$

من القاعدة (3) نلاحظ أن T' ممكن أن تكون ε لذلك $Follow(T)$ إلى $Follow(F)$ أي $\{ \$,), + \}$ ومن القاعدة (4) نلاحظ أن T' هي $First(F)$ و $Follow(F)$ نضيف إليه $First(T')$ أي $\{ * \}$ وبالتالي نلاحظ أن ε لا تضاف إلى $Follow$

ملاحظة مهمة: قبل إيجاد $Follow$ وال $Follow$ يجب أن نقوم بإزالة العودية اليسارية وكذلك إزالة الغموض و سيتم شرح مثال عليها في المحاضرة القادمة إن شاء الله...

بعد إيجاد $Follow$ وال $Follow$ الآن نقوم ببناء جدول ال LL:

خطوات بناء الجدول وملئ الحقول



الفيديو التالي توضيحي

- نقوم ببناء جدول عدد سطوره يساوي عدد القواعد الموجودة +1 وعدد الأعمدة بكل سطر يساوي عدد ال terminal +1
- العمود الأول من السطر الأول يبقى فارغ أما بقية أعمدة السطر الأول فنضع فيها ال terminal التي ظهرت في القواعد.
- بدءاً من السطر الثاني إلى آخر السطر نقوم بملئ العمود الأول من كل سطر باسم القواعد أي فرضاً كان لدي 4 قواعد عندئذٍ أضع بالسطر الثاني R_1 والسطر الثالث R_2 وهكذا...

شكل الجدول الآن:

	t_1	t_2	...	t_n
R_1			...	
R_2			...	
R_3			...	
R_4			...	

- بعد بناء الجدول نقوم بجلب قيمة ال $First$ للقواعد بدايةً من R_1 ثم ملئ الخلايا المقابلة لسطر القاعدة التي تحقق قيمة ال terminal تساوي أحد قيم ال $First$ ، مثلاً: $First(R_1) = \{ t_1, t_n \}$ عندئذٍ نقوم بملئ الخلية المقابلة للسطر R_1 والعمود t_1 بالقاعدة التي سببت ظهور ال t_1 ك $First(R_1)$ وكذلك مع t_2 وهكذا....

وفي حال كانت القاعدة تؤدي إلى قيمة ϵ بشكل مباشر عندئذٍ نقوم بجلب الـ Follow لها وملئ الخلايا المقابلة لقيم الـ Follow مع هذه القاعدة وعندئذٍ تكون القاعدة المسببة بتوليد الـ ϵ هي $R_1 \rightarrow \epsilon$ ■ نكرر القاعدة 4 حتى تنتهي جميع الأسطر.

ملاحظة: في حال ظهور أكثر من بديل في نفس الحقل في الجدول السابق يكون هناك خطأ في الحل.

شرح التمرين:

بعد أن وجدنا الـ First والـ Follow لجميع القواعد وتحققنا من عدم وجود عودية يسارية أو غموض نبدأ ببناء جدول الـ LL(1):

	+	*	(	)	id	\$
E			$E \rightarrow TE'$		$E \rightarrow TE'$	
E'	$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$		$E' \rightarrow \epsilon$
T			$T \rightarrow FT'$		$T \rightarrow FT'$	
T'	$T' \rightarrow \epsilon$	$T' \rightarrow * FT$		$T' \rightarrow \epsilon$		$T' \rightarrow \epsilon$
F			$F \rightarrow (E)$		$F \rightarrow id$	

كيف تم على الحقول؟؟

- بدايةً في السطر الأول وضعنا جميع الـ terminals وفي العمود الأول عند بقية الأسطر وضعنا القواعد:
- بالنظر للسطر الثاني في القاعدة E نقوم بجلب الـ $\{ (, id \} = First(E)$ ونقوم بعدها بملئ الخلية المقابلة لـ $'id'$ و $'($ بالقاعدة التي سببت توليدهم
- فلاحظ أن القاعدة $E \rightarrow TE'$ هي السبب لذلك نعوضها في الحقل المقابل
- بالنظر للسطر الثالث ففي القاعدة E' نقوم بجلب $First(E') = \{ +, \epsilon \}$ وبعدها نقوم بملئ الخلية المقابلة $' + '$ بالقاعدة التي سببت توليدها
- فلاحظ أن القاعدة $E' \rightarrow +TE'$ ولكن نلاحظ أيضاً أن $E' \rightarrow \epsilon$ لذلك نقوم بإيجاد الـ Follow وتعويض الحقول المقابلة لقيم الـ $Follow(E') = \{ \$,) \}$ أي الحقل المقابل لـ $\$$ و $)$ نعوضه بالقيمة التي سببت توليده وهي $E' \rightarrow \epsilon$ وهكذا...



والآن للتحقق أيضاً من صحة القواعد والقيام بـ Action ما عند تحققها:

Input	Stack	Action
$id + id * id \$$	$E \$$	
$id + id * id \$$	$TE' \$$	output $E \rightarrow TE'$
$id + id * id \$$	$FT'E' \$$	output $T \rightarrow FT'$
$id + id * id \$$	$idT'E' \$$	output $T \rightarrow id$
$+id * id \$$	$T'E' \$$	match id
$+id * id \$$	$E' \$$	output $T' \rightarrow \epsilon$
$+id * id \$$	$+TE' \$$	output $E' \rightarrow +TE'$
$id * id \$$	$TE' \$$	match $+$

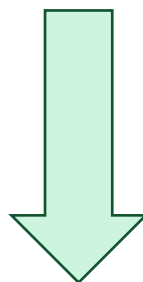
بفرض كان الدخل لدينا " $id + id * id$ " وإشارة \$ تعبر على نهاية السطر.

في البداية يكون المكس فارغ لذلك نضيف له ال start symbol أي القاعدة E والآن نلاحظ أن الدخل لدينا $id + id * id$ والمطلوب التحقق من أن الكلمة تحقق القواعد أو لا.

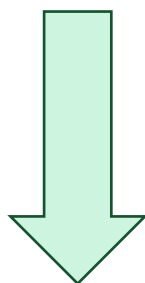
- الآن قمة المكس تساوي E وبداية من الدخل تساوي id وبالعودة الى الجدول السابق LL(1) نلاحظ أن ال Action المناسب عندما تكون القاعدة E وبداية من الدخل id هو $E \rightarrow TE'$ أي نستبدل كل E داخل المكس بـ $E \rightarrow TE'$

- بعد ذلك أصبحت قمة المكس T وبداية من الدخل تساوي id وبالعودة الى الجدول السابق LL(1) نلاحظ أن ال Action المناسب عندما تكون القاعدة T وبداية من الدخل id هو $T \rightarrow FT'$ أي نستبدل كل T داخل المكس بـ $T \rightarrow FT'$

- الآن أصبحت قمة المكس F وبداية من الدخل تساوي id وبالعودة الى الجدول السابق LL(1) نلاحظ أن ال Action المناسب عندما تكون القاعدة F وبداية من الدخل id هو $F \rightarrow id$ أي نستبدل كل F داخل المكس بـ id. الآن نلاحظ أن الدخل $id + id * id \$$ ومحتوى المكس $idT'E \$$ نلاحظ حدوث تطابق بين قمة المكس وبداية الدخل لذلك نقوم بحذفهم وتكرار ماسبق من جديد حتى نحصل على مكس ودخل فارغان عندئذ يكون الدخل محقق للقواعد.



Input	Stack	Action
$id*id\$$	$FT'E'\$$	output $T \rightarrow FT'$
$id*id\$$	$idT'E'\$$	output $F \rightarrow id$
$*id\$$	$T'E'\$$	match id
$*id\$$	$*FT'E'\$$	output $T' \rightarrow *FT'$
$id\$$	$FT'E'\$$	match $*$
$id\$$	$idT'E'\$$	output $F \rightarrow id$
$\$$	$T'E'\$$	match id
$\$$	$E'\$$	output $T' \rightarrow \epsilon$



Input	Stack	Action
$\$$	$\$$	output $E' \rightarrow \epsilon$

سيتم شرح المزيد من الأمثلة في المحاضرة القادمة ..

THE END

